Q1:

Note: The original image is displayed on the left side, while the filtered image is shown on the right side.

Before we start, the comparison of image_i will be against the original image but gray scaled.

Image_1:

What we can notice:

1. The image is represented by horizontal lines.
2. All pixels along the same horizontal line have the same value.
3. The color of the horizontal line in image_1 is about the same as the color of the corresponding horizontal line in the original image.

Considering the 3 remarkable features in image_1, we can conclude that the spatial operation done on the original image to transform it to image_1 is average filter along x axis. Here's why:

From the first observation, we can conclude that the transformation is done along x axis while y axis is untouched.

From the second and third observations, we can conclude that the transformation is the average, since in the average filter that's done horizontally, all the pixels along the same horizontal line will have the same value (because all of them will have the value of the average of the pixels in the original image along the same line).

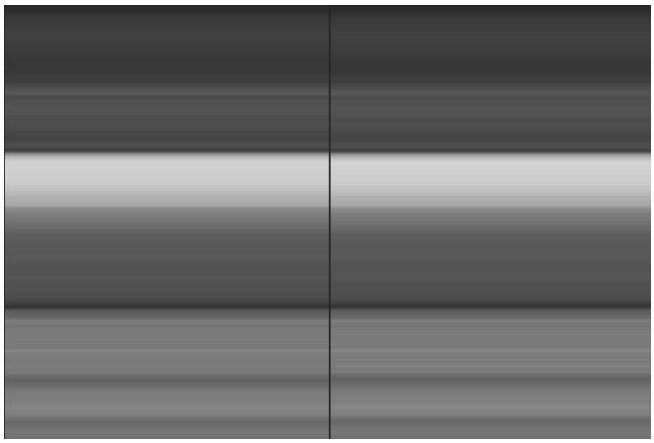


Image_2:

What we can notice:

1. The image is blurry.
2. The edges aren’t handled well.
3. The color of similar regions is about the same.

By these observations, the most reasonable choice is the mean filter since it doesn’t well-handle the edges, make the image blurry and the color around the same location is about the same.

In this image I used a kernel of size 11 x 11.

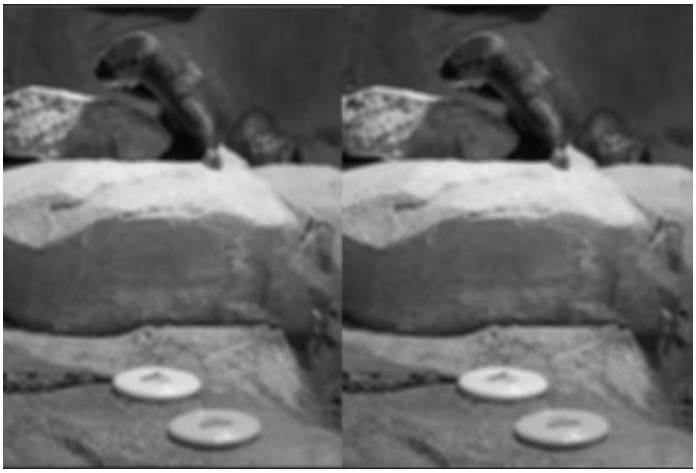


Image_3:

Unlike the previous image, the edges are well-handled, and the colors are not "blended" so the transition between high contrasts areas is sharper. Therefore, I chose the median filter.

In this image I used a kernel of size 11 x 11.

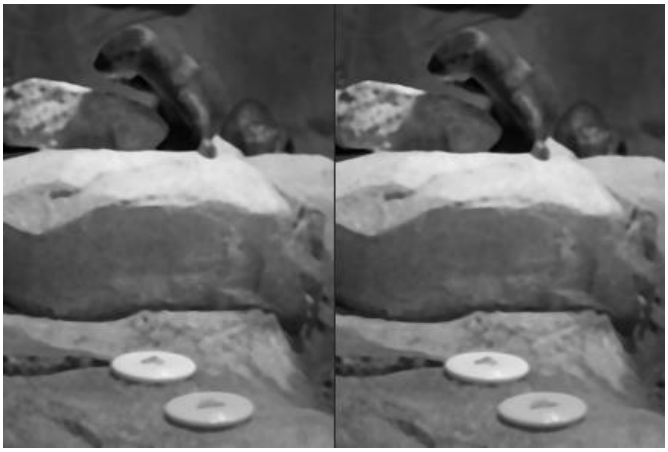


Image_4:

What we can notice:

1. The image look like it has been "shaked" toward y-axis.
2. The horizontal edges are not well-handled.

From these observations, we can conclude that the filter being applied is the mean filter and the window is defined as vertical rectangle.

In this image I used a kernel of size 17 x 17.

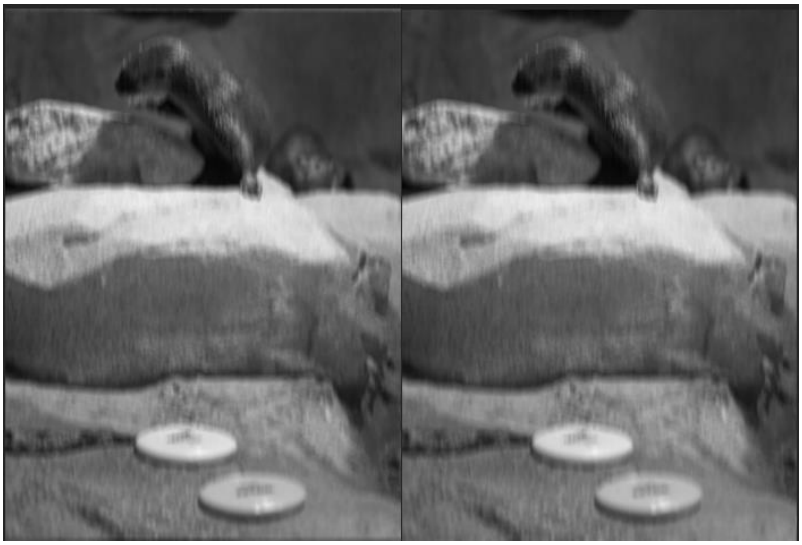


Image_5:

In this image, it looks like when we want to sharpen the image, so we take the sharp part by subtracting from the image its blurry gaussian part.

The edges are more prominent than other parts hence sharper. Therefore, in my opinion, the most logical choice is gaussian filter.

```
def image_5_filter(image):
    return image - cv2.GaussianBlur(image, ksize: (13, 13), sigmaX: 4.4) + 128
```



Image_6:

This filter is the Laplacian filter, since only the edges appear in the image as we have seen in the tutorial.

The mask was:

$$\frac{1}{3} \cdot \begin{bmatrix} -1.1 & -1.1 & -1.1 \\ 0 & 0 & 0 \\ 1.1 & 1.1 & 1.1 \end{bmatrix}$$

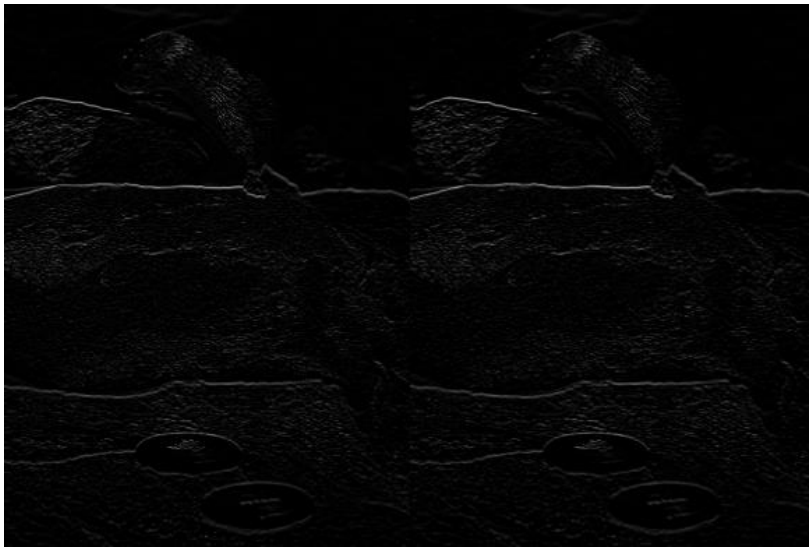


Image_7:

As we can see, the image is being wrapped in cyclic way toward the bottom of y-axis, where the top and bottom halves switch places. Hence, the filter is vertical wrapping.

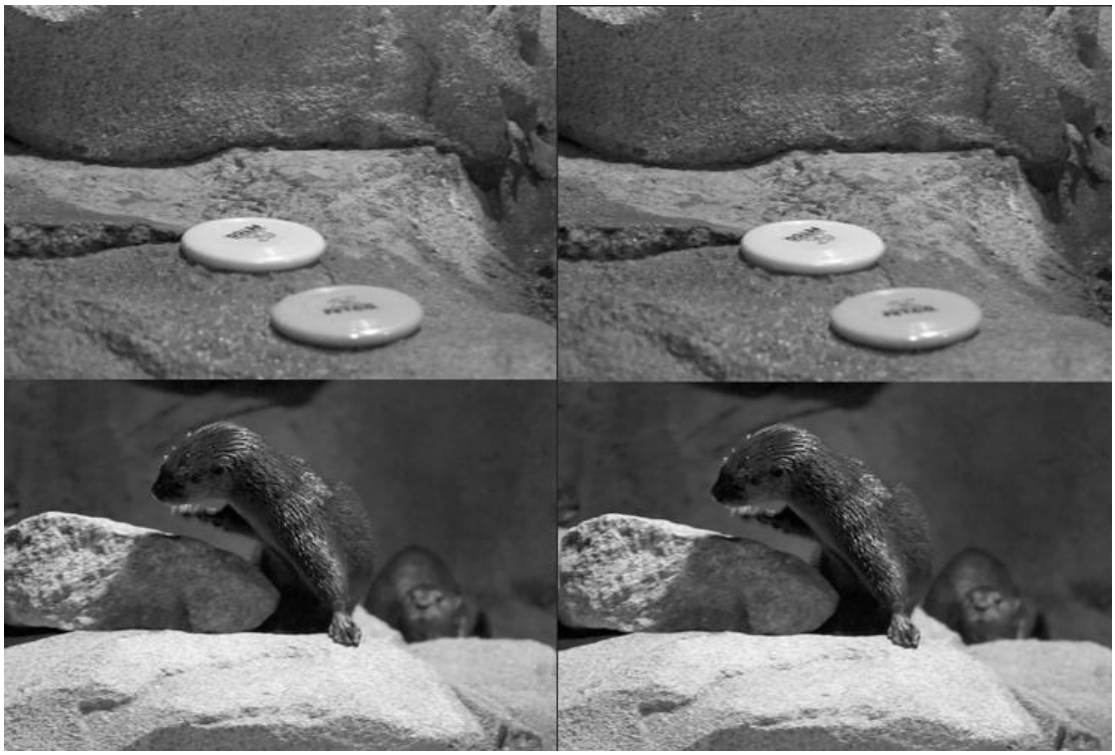


Image_8:

In this image the only thing that's changed is the colors become gray instead of RGB.

So, the filter being applied to the image is changing the colors of the image making it gray scale.

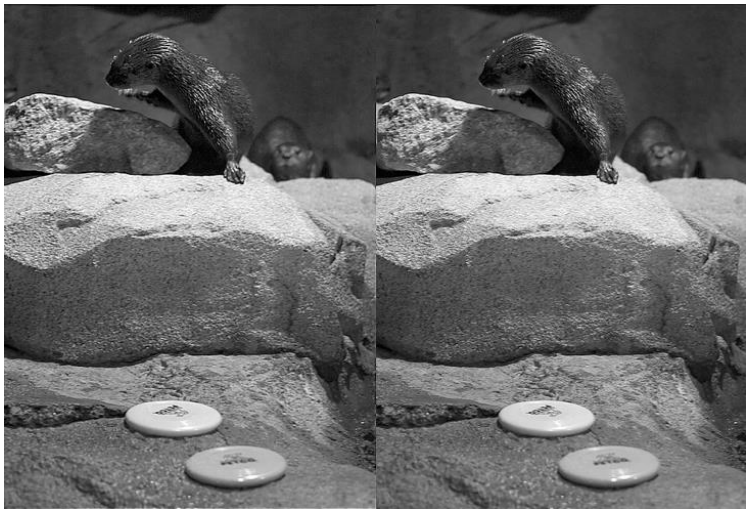


Image_9:

The image got sharper, hence, the filter that's being applied to the image is sharpening filter.

The mask was:

$$\begin{bmatrix} 0 & -0.6 & 0 \\ -0.6 & 3.401 & -0.6 \\ 0 & -0.6 & 0 \end{bmatrix}$$



The MSE:

```
the MSE between image_1 and the filtered is: 0.5175438523292542
the MSE between image_2 and the filtered is: 9.608294486999512
the MSE between image_3 and the filtered is: 0.3677796721458435
the MSE between image_4 and the filtered is: 12.251411437988281
the MSE between image_5 and the filtered is: 10.70022201538086
the MSE between image_6 and the filtered is: 6.194070339202881
the MSE between image_7 and the filtered is: 1.1823054552078247
the MSE between image_8 and the filtered is: 0.34679391980171204
the MSE between image_9 and the filtered is: 16.58694839477539
```

Q2:

a.

Before we start the coding process, I would like to show you how I approach this problem.

First of all, we should understand the goal that we want to achieve. In our case, the goal is to apply the bilateral filter on the image im and produce the output image om. That is,

For each pixel $i, j \in im$:

$$om(i,j) = \frac{\sum_{(x,y)\in N_{i,j}} e^{-\left(\frac{(i-x)^2+(j-y)^2}{\sigma_s^2}\right)} \cdot e^{-\left(\frac{(im[i,j]-im[x,y])^2}{\sigma_p^2}\right)} \cdot im[x,y]}{\sum_{(x,y)\in N_{i,j}} e^{-\left(\frac{(i-x)^2+(j-y)^2}{\sigma_s^2}\right)} \cdot e^{-\left(\frac{(im[i,j]-im[x,y])^2}{\sigma_p^2}\right)}}$$

Hence, we used 3 functions:

1. Extract_window: this function takes array, center and radius and outputs the window of size at most $(2r+1) \times (2r+1)$ depending on the bounds of the array with the center point in the center.

```
x_c, y_c = center
rows, cols = array.shape

# Define the boundaries of the window
x_min = max(0, x_c - radius)
x_max = min(rows, x_c + radius + 1)
y_min = max(0, y_c - radius)
y_max = min(cols, y_c + radius + 1)

# Extract the window
window = array[x_min:x_max, y_min:y_max]
return window, x_min, x_max, y_min, y_max
```

   To make sure not to get out of the bound, we bound the working area by using min and max functions.
   In this code, we handle the bounds of the image and make sure not to get out of bound.

2. Compute_weights: this function returns the spatial weight and intensity weight for the window.

```
# Get the coordinates of the center point
i, j = center

# Get the relative coordinates in the window
x_coords, y_coords = np.meshgrid(
    np.arange(x_min, x_max) - i,
    np.arange(y_min, y_max) - j,
    indexing='ij'
)

# Compute the first expression
spatial_weights = np.exp(-(x_coords**2 + y_coords**2) / (2* std_s**2))

# Compute the second expression
intensity_weights = np.exp(-(window - array[i, j])**2 / (2* std_i**2))
return spatial_weights, intensity_weights
```

Here, we used np.meshgrid to avoid looping over the image making it expensive. So, by using np.meshgrid, we get the pairwise subtraction of between the center point coordinates and the neighbors in the window coordinates.

3. Bilateral_filter: this function returns the filtered image by running over the original image, and for each pixel it extracts the window and calculates the weights then in the output image it calculates the value using the previous formula in the corresponding index. Which is equivalent to this code:

```
array = array.astype(np.float64)
filtered_array = np.zeros_like(array, dtype=float)

for i in range(array.shape[0]):
  for j in range(array.shape[1]):
    center = (i, j)
    window, x_min, x_max, y_min, y_max = extract_window(array, center, radius)
    spatial_weights, intensity_weights = compute_weights(array, center, window, x_min, x_max, y_min, y_max, std_s, std_i)
    filtered_array[i, j] = np.sum(spatial_weights * intensity_weights * window) / np.sum(spatial_weights * intensity_weights)
return filtered_array
```

Note: when we extract the window, we get that the center point is included in the window although we want the neighbors. Hence, if we want to follow the formula, we'll have to subtract the current pixel times $e^0 \cdot e^0$ from the sum. Thus, the updated line will look like:

```
filtered_array[i, j] = (np.sum(spatial_weights * intensity_weights * window) - array[i][j]) / np.sum(spatial_weights * intensity_weights)
```

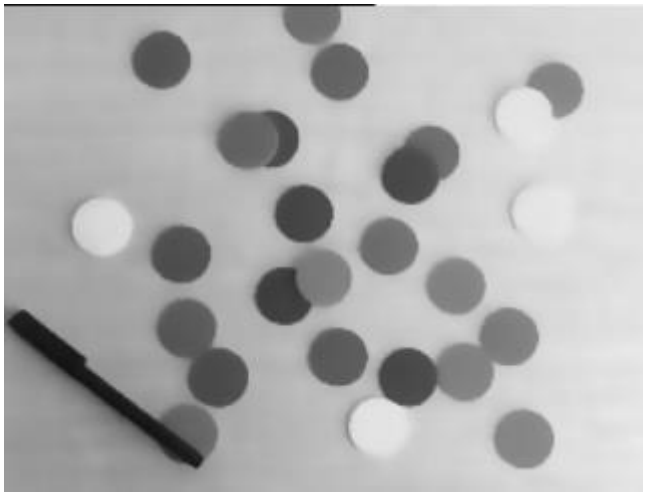
These were the parameters that we used:

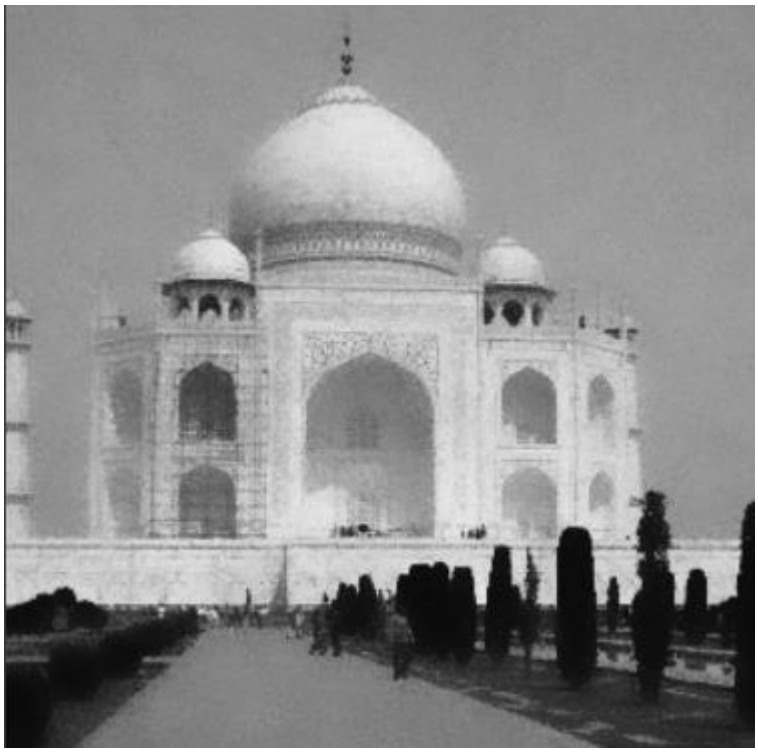
```
paths = ['HW3/q2/NoisyGrayImage.png', 'HW3/q2/balls.jpg', 'HW3/q2/taj.jpg']
radiuses = [9, 3, 9]
std_s = [75, 8, 6]
std_i = [75, 25, 35]
```

The reason why we ended up with those parameters is because we tried several others and those in our opinion were the most suitable.

And these were the results:







I think the filter did a good job overall as in the first image it cleared the noise completely, as well as the second and third. But if there's something that I didn't like it would be that it

reduces some of the details in the image and didn't handle the edges as expected (which we can see in the second image).

Q3:

a. I used a couple of filters on the broken image to get somewhat acceptable result. The filters I used were:
   1. Median filter with kernel size of 9.
   2. Gaussian filter with kernel size of 5.
   3. 3 times median filter with kernel size of 9.
   4. bilateral filter with kernel size of 9, std_i = 75, std_s = 75.

   These filters gave a blurry image but not high quality one, hence I tried some sharpening techniques, but it didn't help. This is the result:



b. By the law of large numbers, we can create a high-quality image M by:

$$M[i,j] = \frac{\sum_{im \in file} im[i,j]}{|file|}$$

   Which is basically the mean filter for the npy file. We did so by using:

   np.mean(noised_images, axis=0)

   and we got the following:

